

Tensor Canonical Polyadic Decomposition and the Complexity of Matrix Multiplication

Tesi di laurea triennale

AUTHOR

25 Settembre 2026

Università di Pisa

1. Applicazione alla complessità aritmetica

Definizione 1.1. Un **algoritmo di moltiplicazione veloce** è un algoritmo che usa asintoticamente un numero minore di moltiplicazioni rispetto alle addizioni.

Definizione 1.2 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , **la moltiplicazione tra due matrici** $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle : \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p}$$
$$(A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo «algoritmo».

1.1 Idea:

Possiamo descrivere un algoritmo di moltiplicazione con un tensore e sperare di ridurre la complessità aritmetica.

1.2 Bilinear computation

Data una mappa bilineare $\varphi : U \times V \rightarrow W$, per ogni $r = 1, \dots, R$ siano $f_r \in U^*$, $g_r \in V^*$, $w_r \in W$ tali che

$$\varphi(u, v) = \sum_{r=1}^R f_r(u)g_r(v)w_r,$$

per ogni $u \in U, v \in V$.

La r -upla $(f_1, g_1, w_1, \dots, f_r, g_r, w_r)$ è detta **bilinear computation**.

1.2 Bilinear computation

Data una mappa bilineare $\varphi : U \times V \rightarrow W$, per ogni $r = 1, \dots, R$ siano $f_r \in U^*$, $g_r \in V^*$, $w_r \in W$ tali che

$$\varphi(u, v) = \sum_{r=1}^R f_r(u)g_r(v)w_r,$$

per ogni $u \in U, v \in V$.

La r -upla $(f_1, g_1, w_1, \dots, f_r, g_r, w_r)$ è detta **bilinear computation**.
La lunghezza di una bilinear computation è detta **bilinear complexity (o rango)** di φ , e si indica con $R(\varphi)$.

1.3 Rappresentazione low-rank di forme bilineari

Proposizione 1.3.1. *Data una decomposizione*

$$T = \sum_{r=1}^R a_r \otimes b_r \otimes c_r$$

Possiamo scrivere una forma bilineare $\varphi : U \times V \rightarrow W$ come una bilinear computation della forma

$$\varphi(u, v) = \sum_{r=1}^R (a_r^\top u) (b_r^\top v) c_r.$$

1.4 Rappresentazione low-rank del prodotto matriciale

Date due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times p}$, poste $u = \text{vec}(A)$, $v = \text{vec}(B)$ abbiamo un algoritmo di moltiplicazione veloce a partire da una bilinear computation

$$\text{vec}(C^\top) = \varphi(u, v) = \sum_{r=1}^R \underbrace{(a_r^\top \text{vec}(A))}_{f_r} \cdot \underbrace{(b_r^\top \text{vec}(B))}_{g_r} c_r,$$

Active multiplications

Conseguenza: $\mathbf{R}(\langle n, m, p \rangle) =$

"Numero di moltiplicazioni matriciali".

1.5 Il rango (cambiare)

Vorremmo tenere traccia della complessità ogni volta che troviamo una decomposizione di $\langle n, n, n \rangle$ di rango minore al variare di n .

1.6 L'esponente

Definizione 1.6.1. Un **circuito aritmetico** Γ è un grafo diretto, orientato, aciclico, finito, con vertici di grado entrante 0, 2 ed esattamente un vertice di grado uscente 0.

- I vertici di grado entrante 0 hanno etichette gli elementi di $\mathbb{K} \cup \{x_1, \dots, x_n\}$, e sono detti **inputs**.
- I vertici di grado entrante 2 hanno etichette $+$ oppure $*$ e sono detti **gates**.
- Se il grado uscente di un vertice è 0 è detto **output**.

1.6 L'esponente

Definizione 1.6.2 (Funzione costo). Sia una mappa bilineare $\varphi : U \times V \rightarrow W$, sia Γ un circuito aritmetico. La **funzione costo** associata a Γ è definita come

$$C_{\Gamma}(\varphi) = \# \text{moltiplicazioni per calcolare } \varphi \text{ su } \Gamma,$$

$$C_{\Gamma}^{\text{tot}}(\varphi) = \# \text{moltiplicazioni e addizioni per calcolare } \varphi \text{ su } \Gamma,$$

Definizione 1.6.3 (Complessità aritmetica). Fissato $\mathcal{C}_{\varphi} = \{\text{circuiti } \Gamma' \text{ che calcolano } \varphi\}$, definiamo

$$L(\varphi) = \inf_{\Gamma \in \mathcal{C}_{\varphi}} C_{\Gamma}(\varphi), \quad \text{la } \textbf{complessità moltiplicativa} \text{ di } \varphi,$$

$$L^{\text{tot}}(\varphi) = \inf_{\Gamma \in \mathcal{C}_{\varphi}} C_{\Gamma}^{\text{tot}}(\varphi), \quad \text{la } \textbf{complessità totale} \text{ di } \varphi.$$

Definizione 1.6.4. Denotiamo il **numero di moltiplicazioni aritmetiche** necessarie a calcolare $\langle n, n, n \rangle$ con

$$M_{\mathbb{K}}(n) = L^{\text{tot}}(\langle n, n, n \rangle)$$

Definizione 1.6.5. L'**esponente della moltiplicazione matriciale** è il numero

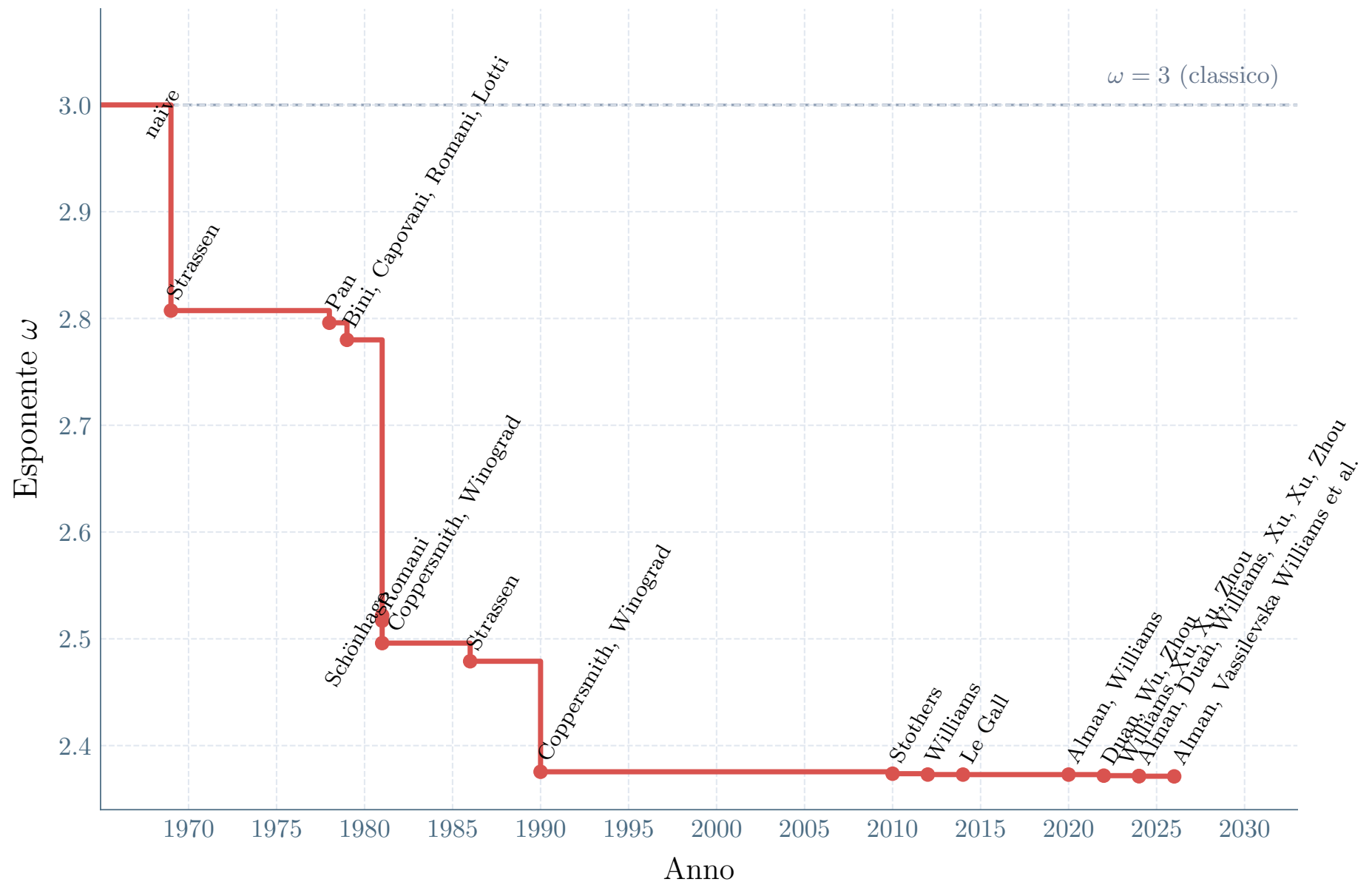
$$\begin{aligned} \omega &:= \inf_n \{ \tau \in \mathbb{R} \mid \text{La moltiplicazione tra due matrici in } \mathbb{R}^{n \times n} \\ &\quad \text{ha costo } o(n^\tau). \} \\ &= \inf_n \{ \tau \in \mathbb{R} \mid M_{\mathbb{K}}(n) = o(n^\tau). \} \end{aligned}$$

Osservazione 1.6.6. $\omega \leq 3$

Congettura 1.6.7. $\omega = 2$

1. Applicazione alla complessità aritmetica

1.6.1 Stime note su ω



1.7 L'algoritmo classico

$$\begin{aligned} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \\ &= \begin{bmatrix} f_1g_1 + f_2g_2 & f_3g_3 + f_4g_4 \\ f_5g_5 + f_6g_6 & f_7g_7 + f_8g_8 \end{bmatrix} \\ &= \begin{bmatrix} m_1 + m_2 & m_3 + m_4 \\ m_5 + m_6 & m_7 + m_8 \end{bmatrix} \\ &= \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \end{aligned}$$

Fissata la base standard $E_{11}, E_{12}, E_{21}, E_{22}$ dello spazio delle matrici 2×2 , possiamo scrivere la bilinear computation

$$\begin{aligned} C &= c_{11}E_{11} + c_{12}E_{12} + c_{21}E_{21} + c_{22}E_{22} \\ &= (m_1 + m_2)E_{11} + (m_3 + m_4)E_{12} \\ &\quad + (m_5 + m_6)E_{21} + (m_7 + m_8)E_{22} \\ &= \sum_{r=1}^8 m_r w_r. \end{aligned}$$

L'algoritmo corrisponde al tensore

$$\begin{aligned} \langle 2, 2, 2 \rangle &= (a_{11} \otimes b_{11} + a_{12} \otimes b_{21}) \otimes c_{11} \\ &\quad + (a_{21} \otimes b_{11} + a_{22} \otimes b_{21}) \otimes c_{21} \\ &\quad + (a_{11} \otimes b_{12} + a_{12} \otimes b_{22}) \otimes c_{12} \\ &\quad + (a_{21} \otimes b_{12} + a_{22} \otimes b_{22}) \otimes c_{22}. \end{aligned}$$

1.8 L'algoritmo di Strassen

Facendo alcune semplificazioni del risultato del prodotto matriciale, troviamo l'algoritmo di Strassen in forma tensoriale

$$\begin{aligned}
 \langle 2, 2, 2 \rangle = & (a_{11} + a_{22}) \otimes (b_{11} + b_{22}) \otimes (c_{11} + c_{22}) \\
 & + (a_{21} + a_{22}) \otimes b_{11} \otimes (c_{21} - c_{22}) \\
 & + a_{11} \otimes (b_{12} - b_{22}) \otimes (c_{12} + c_{22}) \\
 & + a_{22} \otimes (-b_{11} + b_{21}) \otimes (c_{21} + c_{11}) \\
 & + (a_{11} + a_{12}) \otimes b_{22} \otimes (-c_{11} + c_{12}) \\
 & + (-a_{11} + a_{21}) \otimes (b_{11} + b_{12}) \otimes c_{22} \\
 & + (a_{12} - a_{22}) \otimes (b_{21} + b_{22}) \otimes c_{11}.
 \end{aligned}$$

Che mostra $\mathbf{R}(\langle 2, 2, 2 \rangle) = 7$.

1.9 Approccio divide et impera

La formula

$$\text{vec}(C^\top) = \sum_{r=1}^R (a_r^\top \text{vec}(A)) \cdot (b_r^\top \text{vec}(B)) c_r,$$

si può utilizzare in maniera ricorsiva.

Idea: Si applica la formula ricorsivamente su $\cdot$ fino al caso base.

Due domande:

Costo computazionale della ricorsione?

Come ridurre il rango migliora ω ?

1.10 Stimare la complessità nel caso generale

Date due matrici di taglia $2^i \times 2^i$ possiamo fare i livelli di ricorsione e ottenere che

$$R(\langle 2^i, 2^i, 2^i \rangle) = R(\langle 2, 2, 2 \rangle^{\otimes i}) \leq R(\langle 2, 2, 2 \rangle)^i \leq r^i.$$

Nel caso generale, dato $n > 2$, possiamo fare un padding di zeri

$$\begin{bmatrix} & & & 0 \\ & A & & 0 \\ & & & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} & & & 0 \\ & B & & 0 \\ & & & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

fino a raggiungere la taglia $2^{\lceil \log_2 n \rceil}$.

La complessità sarà comunque limitata da R :

$$\begin{aligned} \mathbf{R}(\langle n, n, n \rangle) &\leq \mathbf{R}(\langle 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil} \rangle) \\ &\leq \mathbf{R}(\langle 2, 2, 2 \rangle)^{\lceil \log_2 n \rceil} \\ &\leq R^{\lceil \log_2 n \rceil} \\ &\leq R \cdot n^{\log_2 R} \end{aligned}$$

Da cui (sostituendo 2 con $\hat{n}$)

Proposizione 1.10.1. *Se $\mathbf{R}(\langle \hat{n}, \hat{n}, \hat{n} \rangle) \leq R$ per degli interi positivi n, R , allora $\hat{n}^\omega \leq R$.*

Abbiamo dimostrato che $\omega \leq \log_n \mathbf{R}(\langle n, n, n \rangle)$.

Da cui,

$$\omega \leq \inf_n \log_n \mathbf{R}(\langle n, n, n \rangle).$$

1.10.1 Applicazione all'algoritmo di Strassen

Se $R = 8$, dalla Proposizione 1.10.1 troviamo

$$\omega \leq \log_2 \mathbf{R}(\langle 2, 2, 2 \rangle) = \log_2 8 = 3.$$

L'algoritmo di Strassen migliora questa stima in quanto

$$\omega \leq \log_2 \mathbf{R}(\langle 2, 2, 2 \rangle) = \log_2 7 \approx 2.81.$$

Che è il bound più basso per ω ottenibile da $\langle 2, 2, 2 \rangle$.

Ad esempio, Bini et al. [1] trovano che $\mathbf{R}(\langle 12, 12, 12 \rangle) = 100$, da cui

$$\omega \leq \log_{12} 1000 \approx 2.7799.$$

1.11 Il teorema di Strassen

Vorremmo che

$$\omega = \liminf_{n \rightarrow +\infty} \log_n \mathbf{R}(\langle n, n, n \rangle).$$

Teorema 1.11.1 (Strassen [2], [3]).

$$\omega = \inf_n \{ \tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau) \}.$$

Ovvero, moltiplicare due matrici $n \times n$ costa $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se $\mathbf{R}(\langle n, n, n \rangle) = O(n^{\omega+\varepsilon})$.

1.12 Algoritmi approssimati

Bini et al. CITE partendo dall'algoritmo classico $\langle 2, 2, 2 \rangle$ e ponendo l'entrata $a_{22} = 0$,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} \end{bmatrix}.$$

Trovano il tensore di rango 6

$$\begin{aligned} \langle 2, 2, 2 \rangle^{\text{red}} &:= a_{11} \otimes (b_{11} \otimes c_{11} + b_{12} \otimes c_{12}) \\ &\quad + a_{12} \otimes (b_{21} \otimes c_{11} + b_{22} \otimes c_{12}) \\ &\quad + a_{21} \otimes (b_{11} \otimes c_{21} + b_{12} \otimes c_{22}). \end{aligned}$$

Nel tentativo di ridurre numericamente il rango a 5, trovano un tensore

$$\begin{aligned}
 \langle 2, 2, 2 \rangle_t^{\text{red}} = & (a_{12} + ta_{11}) \otimes (b_{12} + tb_{22} \otimes c_{12}) \\
 & + (a_{21} + ta_{11}) \otimes b_{11} \otimes (c_{11} + tc_{21}) \\
 & + a_{12} \otimes b_{12} \otimes ((c_{11} + c_{12}) + tc_{22}) \\
 & + a_{21} \otimes ((b_{11} + b_{12}) + tb_{21}) \otimes c_{11} \\
 & + (a_{12} + a_{21}) \otimes (b_{12} + tb_{21}) \otimes (c_{11} + tc_{22})
 \end{aligned}$$

Scoprendo (accidentalmente) che

$$\lim_{t \rightarrow 0} \langle 2, 2, 2 \rangle_t^{\text{red}} = \langle 2, 2, 2 \rangle^{\text{red}}$$

Ovvero

$$\underline{R}(\langle 2, 2, 2 \rangle^{\text{red}}) \leq 5$$

Da cui ricavano l'algoritmo $\langle 3, 2, 2 \rangle$ di complessità $3 \log_{12} 10 \approx 2.7799$.

1.13 Confronto numerico

2. Bibliografia

- [1] D. Bini, M. Capovani, F. Romani, e G. Lotti, « $O(n^2)$ complexity for $n \times n$ approximate matrix multiplication», *Information processing letters*, vol. 8, fasc. 5, pp. 234–235, 1979.
- [2] V. Strassen, «Gaussian elimination is not optimal», *Numer. Math.*, vol. 13, fasc. 4, pp. 354–356, ago. 1969, doi: 10.1007/BF02165411.
- [3] P. Bürgisser, M. Clausen, e M. A. Shokrollahi, *Algebraic complexity theory*, vol. 315. Springer Science & Business Media, 2013.